

AI-verktyg i systemutveckling

Så togs prognosfunktionen i SubTracker fram, granskades och förbättrades med AI

Applikation	SubTracker, en prenumerationsöversikt (ASP.NET Core 10 Web API, React/Vite, Azure)
Vald del	Prognosfunktionen: <code>GET /api/forecast</code> och prognoskortet på dashboarden
AI-verktyg	Claude Code (Anthropic)
Repository	https://github.com/HappyHamster135/Fullstack-App-Azure-Ai gren <code>claude/blissful-edison-7izj8g</code>
Commits	Bilaga A och filen inlamning.txt. Loggen med alla prompts och svar finns i <code>docs/ai/</code> .
Datum	4–5 oktober 2026

Innehåll

1. Sammanfattning	2
2. Vald del av applikationen	3
3. AI-verktyg och arbetssätt	5
4. AI-tillfälle 1: kodgenerering	6
5. Min granskning av AI:s kod	7
6. AI-tillfälle 2: oberoende analys	10
7. Test och slutverifiering	12
8. Hur kvalitet, säkerhet och arkitektur kontrollerades	16
9. Bedömning av AI-verktygets bidrag	16
Bilaga A. Commits som hör till uppgiften	17
Bilaga B. Köra verifieringen	18

1. Sammanfattning

Uppgiften var att vidareutveckla SubTracker med ett AI-verktyg, granska det som AI:n genererar och ta ansvar för koden som hamnar i projektet. Jag valde att bygga en **prognosfunktion**: ett nytt API-endpoint (GET /api/forecast) som räknar ut vad användaren förväntas betala per månad framåt, och ett kort på dashboarden med ett stapeldiagram. Delen är avgränsad men går genom alla lager (controller, service, entitet, DTO:er, React) och innehåller två saker som är lätta att få fel: datumberäkningar och indata från användaren.

Som AI-verktyg använde jag **Claude Code** i tre roller: en huvudsession som styrde, granskade, testade och dokumenterade, samt två *separata* agenter med tomt sammanhang. Den ena skrev funktionen (AI-tillfälle 1) och den andra granskade den oberoende (AI-tillfälle 2). Att de var separata var avsiktligt, se avsnitt 3.

Vad som hände

- 1. **AI:ns förslag var till stor del rätt.** Rätt arkitektur på första försöket, korrekt datumlogik (0 avvikelser mot en oberoende beräkning) och användarisolering. Jag committade det orört för att kunna visa vad granskningen ändrade.
- 2. **Min granskning hittade fem förbättringar**, flera av dem först synliga när jag testade, mätte eller körde appen: en klocka som inte gick att styra i test, ett valideringsmönster som skilde sig från resten av projektet, arbete som växte med hur gammalt användarens datum var, onödigt spårning av entiteter och en rubrik som kunde missförstås.
- 3. **Den oberoende granskningen (AI-tillfälle 2) hittade 12 fynd till** (2 medel, 10 låga). Jag verifierade alla själv och åtgärdade 11 helt eller delvis. Det tolfte, ett beteende som kräver en migration, lämnade jag medvetet.
- 4. **Jag tillämpade granskarens egen metod, mutationsprovning, på mina egna rättningar** och hittade ett hål där, som jag stängde.

Mått	Före	Efter
Backendtester	4	80
Frontendtester	0 (ingen testkörare)	33
Tester körs i CI före deploy	nej	ja (backend och frontend)
Granskarens 29 trasiga kodvarianter som testerna fångar (mutationsprovning)	19 av 29	29 av 29
Oberoende beräkning mot API:t (260 prenumerationer, 1–24 månader)	0 avvikelser	0 avvikelser
Scriptad användarresa i webbläsare	–	18 av 18 kontroller
Svarsstorlek, realistisk användare, 12 månader	54,4 KiB	4,5 KiB
Värsta svaret för ett konto som missbrukar API:t	22 MiB, växande	0,01 MiB (4,4 MiB med lista)
"Idag" klockan 23:30 UTC (00:30 svensk tid)	fel dag	rätt dag
Diagrametiketter som överlappar på 360 px	5	0

Vem gjorde vad. Hela arbetet, inklusive granskningarna och bedömningarna, utfördes av Claude Code på mitt uppdrag i en session som jag startade. "Jag" i texten avser den sessionen. Prompterna till de två AI-agenterna står ordagrant i avsnitt 4 och 6, och allt som påstås här går att kontrollera i repots commits och i docs/ai/ .

2. Vald del av applikationen

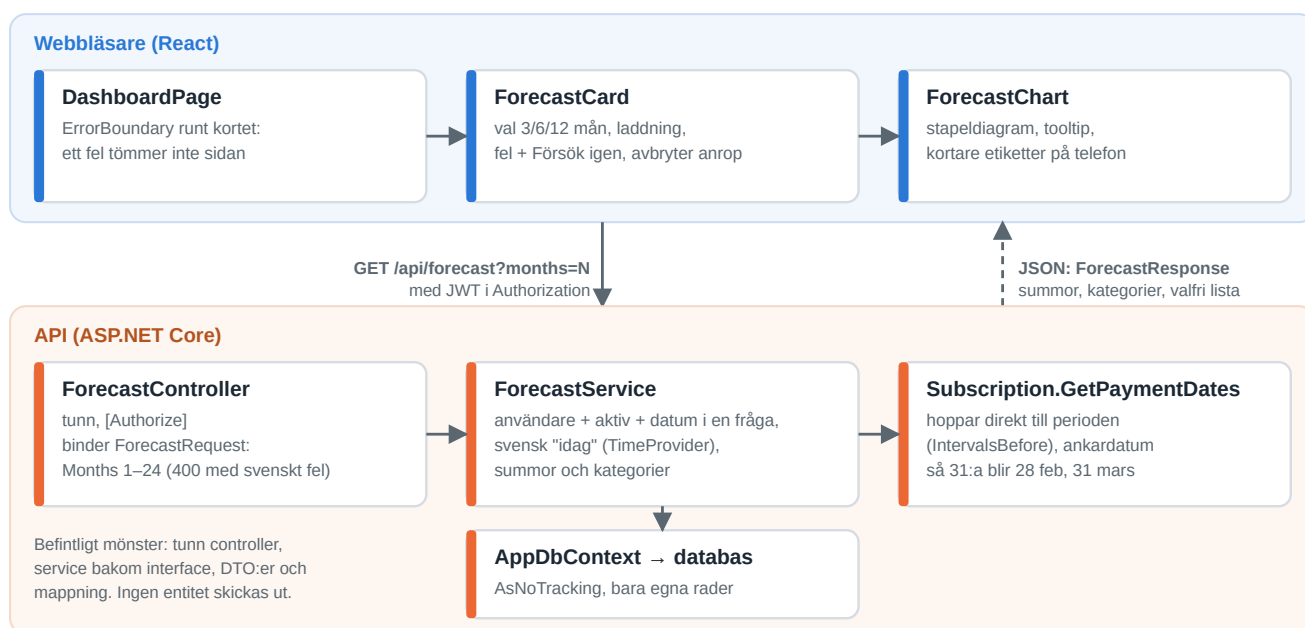
SubTracker är en webbapp där användare registrerar sina digitala prenumerationer (pris, kategori, betalningsintervall) och får en dashboard med kostnader. Backend är ASP.NET Core 10 med EF Core, Identity och JWT. Frontend är React med Vite. Arkitekturen är genomgående: tunna controllers, affärslogik i services bakom interface, DTO:er och mappningar mellan API och entiteter.

Funktionen

- `GET /api/forecast?months=N` (inloggning krävs, N är 1–24 och standard är 6) räknar ut förväntade betalningar för användarens aktiva prenumerationer utifrån nästa betalningsdatum och intervall (vecka, månad, kvartal, år).
- Svaret ger per månad summa, fördelning per kategori och, om det efterfrågas, de enskilda betalningarna, samt en total för perioden.
- Dashboarden får ett kort med stapeldiagram och val av 3, 6 eller 12 månader.

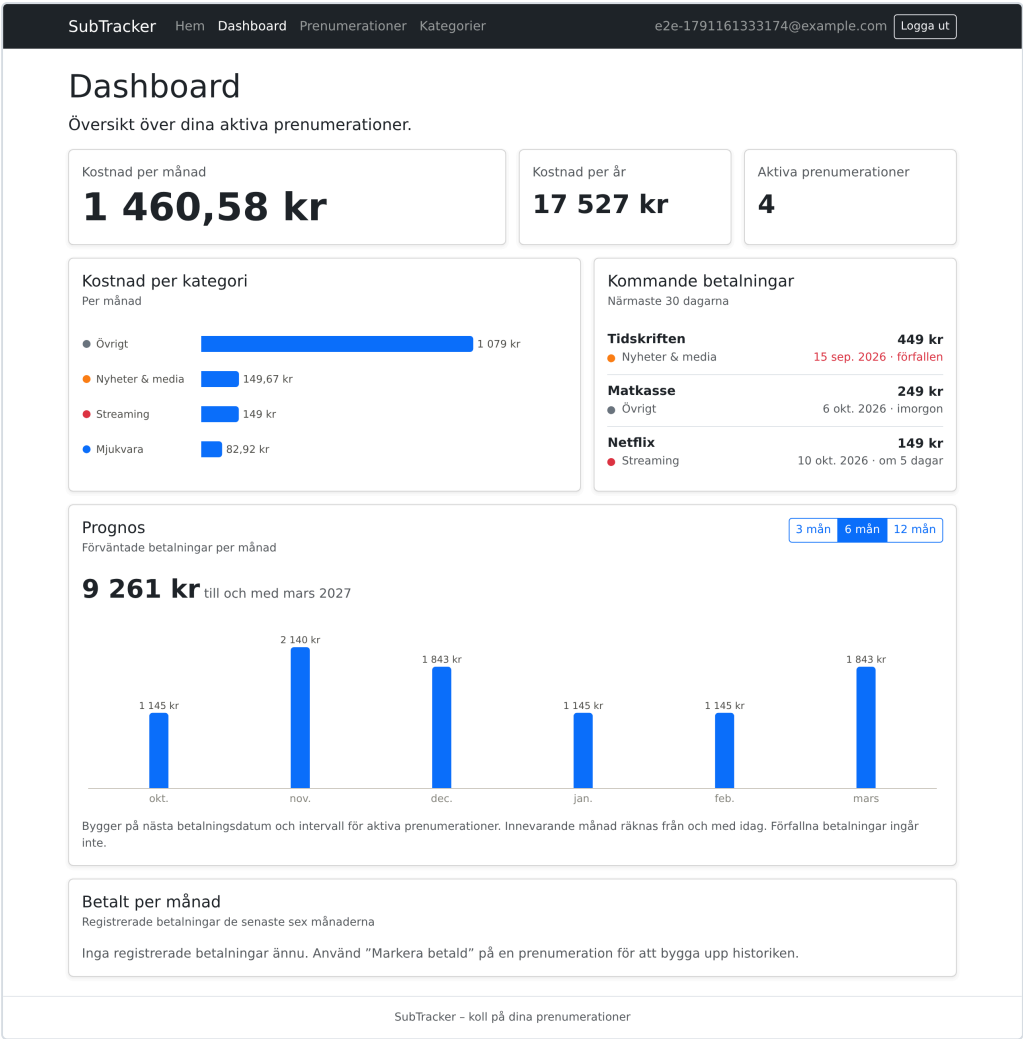
Varför jag valde just den här delen

- **Den berör alla lager**, så det går att bedöma om AI:n följer projektets arkitektur och ansvarsfördelning.
- **Den innehåller datumlogik** (månadsskiften, skottår, förfallna betalningar), som är lätt att få subtilt fel och svårt att se vid läsning.
- **Den tar indata från användaren** (antal månader, datum, antal prenumerationer), vilket gör säkerhet, validering och resursförbrukning relevanta.
- **Den går att kontrollera objektivt**: svaret kan jämföras med en oberoende beräkning.



Figur 1. Prognosfunktionen genom lagren. Blått är frontend, orange är API:t.

Lager	Filer (nya om inget annat anges)
Controller	ForecastController
Service	IForecastService, ForecastService, SwedishTimeProvider och TimeProviderExtensions (svensk "idag")
DTO:er och mappning	ForecastRequest, ForecastResponse, ForecastMonthResponse, ForecastCategoryResponse, ForecastPaymentResponse, ForecastMappings
Domän	Subscription.GetPaymentDates och BillingInterval.AddIntervals / IntervalsBefore (befintliga filer utökades)
Frontend	ForecastApi, ForecastCard, ForecastChart, ErrorBoundary och integration i DashboardPage
Tester	Nytt projekt SubTracker.Api.Tests (xUnit) och Vitest i frontend/



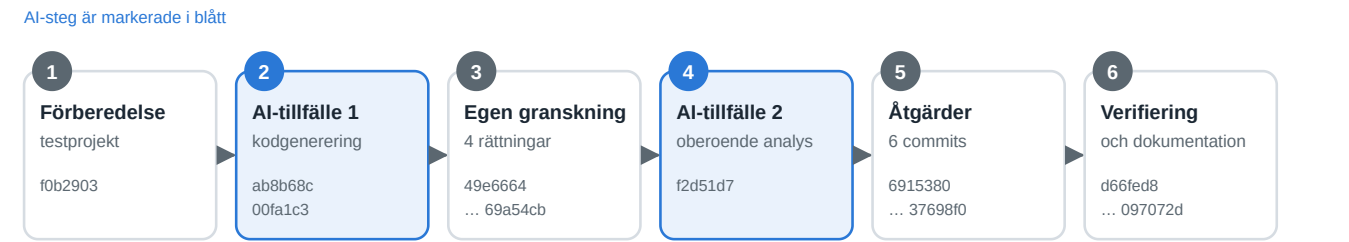
Figur 2. Dashboarden med prognoskortet (längst ned). Skärmdump ur webbläsarkörningen mot det riktiga API:t.

3. AI-verktyg och arbetssätt

Claude Code är ett AI-verktyg för programmering som arbetar direkt i ett repo: det läser och skriver filer, kör kommandon (bygge, tester) och kan styra en webbläsare. Jag använde det i en molnsession med tillgång till repot, .NET, Node och en Chromium.

Roll	Vem	Uppgift	Fick se
Styrning, granskning, tester, rättningar, dokumentation	Claude Code, huvudsessionen	Allt som inte är de två AI-tillfällena	Allt
AI-tillfälle 1: generera koden	Separat Claude-agent, tomt sammanhang	Bygga funktionen efter prompten	Prompten och repot
AI-tillfälle 2: analysera och felsöka	Separat Claude-agent, tomt sammanhang	Granska den rättade koden och bevisa fynd, utan att ändra något i repot	Prompten och repot. Fick inte läsa mina granskningsanteckningar

Varför separata agenter? Om samma session skriver koden och sedan granskar den är det svårt att se sina egna antaganden. När agenterna startar utan sammanhang blir förslaget oretuscherat, och den andra bedömningen påverkas inte av min egen granskning. Det gör också dokumentationen ärlig: det som står om vad AI:n föreslog är vad den faktiskt föreslog.



Figur 3. Arbetets steg och commit-hashar (alla hashar finns i bilaga A). AI-tillfällena är markerade i blått.

Regler jag arbetade efter

- **Lita inte på AI:ns egna försäkringar.** Att AI:n skriver "kompilerar och testat" ersätter inte att köra själv.
- **Committa förslaget orört** innan något ändras, så att diffen visar exakt vad granskningen förändrade.
- **Bevisa med körning:** tester, mätning, jämförelse mot en oberoende beräkning, mutationsprovning och webbläsare. Inte bara läsning.
- **Skriv testet först** och se det vara rätt mot den gamla koden innan rättningen.
- **En rättning per commit** med vad som var fel, beläggen och vad som ändrades.
- **Dokumentera det som inte åtgärdas**, med skäl. Att granska är också att avgöra vad som inte ska ändras.

En störning att känna till. Medan den första granskningsagenten arbetade startades containern som sessionen kör i om. Agentens arbete gick förlorat utan någon rapport, och jag körde om uppgiften med exakt samma prompt. Allt som redovisas i avsnitt 6 kommer från den andra körningen. Det visar att AI-arbete som inte sparas i repot kan försvinna. Själva repot klarade omstarten, och den commit som då ännu inte var pushad pushades direkt efteråt.

4. AI-tillfälle 1: kodgenerering

Prompten gick till en agent med tomt sammanhang. Jag skrev den som en utvecklare skulle skriva den: mål, krav och konventioner, men *inte* fallgorpar. Jag bad uttryckligen att den inte skulle skriva tester ("testerna skriver jag själv") och inte committa.

Prompt (ordagrant)

Jag vidareutvecklar min webbapplikation SubTracker (ASP.NET Core 10 Web API + React/Vite) där användare håller koll på sina digitala prenumerationer. Repot ligger i /home/user/fullstack-app-azure-ai – läs README.md först för att förstå arkitekturen.

Uppgift: bygg en prognosfunktion som visar vad användaren kommer att betala framåt i tiden.

Funktion

- Nytt API-endpoint GET /api/forecast?months=N (kräver inloggning, precis som övriga endpoints). Det räknar fram en prognos för de kommande N månaderna (standard 6) utifrån användarens aktiva prenumerationer. Nästa betalningsdatum och betalningsintervall (vecka/månad/kvartal/år) avgör vilka datum en prenumeration dras.
- Svaret ska för varje månad ge den totala förväntade kostnaden, hur den fördelas per kategori och vilka enskilda betalningar (prenumeration, belopp, datum) som ingår. Det ska också ge en total för hela perioden.
- Frontend: visa prognosen på dashboard-sidan (diagram per månad och möjlighet att välja antal månader, t.ex. 3/6/12) i samma stil som övriga dashboard-komponenter.

Krav

- Följ projektets befintliga arkitektur och konventioner: tunn controller, logik i en service bakom ett interface, DTO:er och mappningar, ServiceResult/ServiceError, svenska felmeddelanden och kommentarer i samma stil som övrig kod, samt React-komponenter och API-klass på samma sätt som idag.
- Skriv ren, läsbar kod och ändra inte befintlig funktionalitet i onödan.
- Kontrollera att det kompilerar: `dotnet build backend/SubTracker.Api/SubTracker.Api.csproj` och, i frontend/, `npm ci && npm run lint && npm run build`.
- Testerna skriver jag själv, så lägg inte till några. Gör inga git-commits och arbeta bara inne i /home/user/fullstack-app-azure-ai.

Svara på svenska med en kort sammanfattning av vad du gjort och vilka designval du gjorde.

Vad AI-verktyget föreslog

Förslaget blev två commits: backend med 12 filer (190 tillagda rader) och frontend med 5 filer (273 tillagda rader). Agenten hävdade att allt kompilerar, linter rent och att de fyra befintliga testerna passerar, vilket jag kontrollerade själv. Dess viktigaste designval:

Designval i AI:ns svar	Min bedömning
Perioden är N hela kalendermånader från innevarande månad, och bara betalningar från och med idag räknas	Rimligt. Rubriken i gränssnittet kunde dock missförstås (rättades, F5).
Förfallna betalningar ingår inte, men intervallet följs vidare	Rimligt, men dashboardens "Kommande betalningar" visar dem. Förtydligades i texten (AI-tillfälle 2, F9).
Bara aktiva prenumerationer. Beloppet är fullt pris på betalningsdagen, inte månadskostnad	Rätt: det är faktiska dragningar.
Inget datumglidande: varje datum räknas från nästa betalning + k intervall (31 jan → 28 feb → 31 mars)	Korrekt. Bevisat med en oberoende beräkning, 0 avvikelser.
months (1–24) valideras i servicen med ett handbyggt fel	Avvek från projektets mönster (rättades, F2).
En databasfråga med filter på användare, aktiv och datum, resten i minnet	Bra. Enkel fråga som översätts likadant till SQL Server och SQLite.

Vilka delar jag använde

Jag använde **allt** i förslaget som utgångspunkt och committade det orört. Efter granskningarna ändrades dessa delar:

Del	Ändrades efter granskningen?
<code>ForecastController</code>	Ja. Binder nu <code>ForecastRequest</code> (F2) och skickar vidare <code>includePayments</code> .
<code>ForecastService</code>	Ja. Styrbar klocka (F1), validering ut ur servicen (F2), <code>AsNoTracking</code> (F4), svensk tid, intern <code>Charge</code> , valfri betalningslista och deterministisk sortering.
DTO:er och <code>ForecastMappings</code>	Behövs. <code>Payments</code> blev valfri och <code>ForecastRequest</code> tillkom.
<code>Subscription.GetPaymentDates</code>	Ja. Hoppar direkt till perioden i stället för att stega från ett gammalt datum (F3). Kommentaren rättades.
<code>BillingInterval.AddIntervals</code> / <code>NextDateAfter</code>	Behövs och utökades med <code>IntervalsBefore</code> (F3). <code>NextDateAfter</code> delegerar till <code>AddIntervals</code> utan ändrat beteende, vilket ett test visar.
<code>ForecastApi</code> , <code>ForecastCard</code>	Ja. Rubrik (F5), avbrytbara anrop, "Försök igen", <code>aria-busy</code> .
<code>ForecastChart</code>	Ja. Kortare etiketter på telefon, tål saknad data, diagrammet har ett namn.
<code>DashboardPage</code> , <code>format.js</code>	Ja. Felgräns runt kortet och formatering av korta belopp.

Min första bedömning

Förslaget byggde, lintade och mina fyra baslinjetester var gröna. Det **såg bra ut**: tunn controller, logiken bakom ett interface, DTO:er, ingen entitet skickas ut, svenska meddelanden, `[Authorize]` och filtrering på användare. Det återanvände till och med `NextDateAfter` i stället för att skriva ny datumlogik. Det betyder inte att det var *rätt*. Beräkningen bygger på datumregler och på användarstyrd data, och sådant är lätt att få fel men svårt att se vid läsning. Jag tog med mig fem frågor till granskningen: går beräkningen att testa när klockan läses inne i servicen, hur mycket arbete kan ett obegränsat datum orsaka, följer valideringen projektets mönster, läser koden mer än den behöver, och kan siffrorna missförstås?

5. Min granskning av AI:s kod

Metod

AI:n skrev själv att allt kompilerar och är testat. Det räckte inte, så jag granskade på sex sätt:

1. **Läste hela diffen** rad för rad och jämförde med projektets mönster.
2. **Byggde och lintade** och körde baslinjetesterna.
3. **Jämförde mot en oberoende beräkning**. Jag skrev en egen beräkning av förväntade datum i Python, utan någon kod gemensam med C#, och körde den mot det riktiga API:t för 260 prenumerationer (31 januari, 29 februari, förfallna datum, alla intervall) och fem periodlängder: **0 avvikelser**.
4. **Skrev tester med fast klocka** genom hela kedjan (JWT, controller, service, EF Core, SQLite).
5. **Mutationsprovning**. Jag förstörde koden med flit på olika sätt och kontrollerade att testerna larmade. Ett test som aldrig kan fela bevisar ingenting.
6. **Mätte och körde** mot det riktiga API:t i en riktig webbläsare.

Checklista: de sex områdena

Område	Bedömning och belägg
Fungerar tillsammans med resten av appen	Ja. Efter första granskningen var 40 tester genom hela kedjan gröna, och differentialtestet gav 0 avvikelser av 260 prenumerationer × 5 periodlängder. Efter båda granskningarna är det 80 backendtester, 33 frontendtester och 18 av 18 kontroller i webbläsare (avsnitt 7). AI:ns ändring av den befintliga <code>NextDateAfter</code> bryter inte "Markera betald" (eget test).
Följer arkitektur och ansvarsfördelning	Till största delen: tunn controller, service bakom interface, DTO:er och datumlogik på entiteten (som <code>RegisterPayment</code>). Två avvikelser, validering i servicen (F2) och en klocka som inte gick att styra (F1), rättades.
Onödig eller duplicerad logik	Bra. AI:n återanvände <code>NextDateAfter</code> via <code>AddIntervals</code> . Onödigt var <code>ServiceResult</code> för ett fel som en DTO hanterar bättre (rättat). <code>ForecastChart</code> liknar <code>PaymentsChart</code> , men datan och tooltipen skiljer sig, så en abstraktion med två användare vore för tidig. Lämnades.
Indata och användardata säkert	Åtkomstkontrollen är rätt: alla frågor filtreras på inloggad användare, och testet går sönder när jag tar bort filtret. Brister kring indata: <code>months</code> hade fel felformat (F2), användarens datum styrde hur mycket arbete en prenumeration kostade (F3), och datum och antal prenumerationer saknade gränser (se avsnitt 6).
Secrets och känslig information	Inga. Sökning efter lösenord, nycklar, token, connection strings och URL:er i produktionskoden ger bara README-text och en befintlig rad. Inga konfigurationsfiler ändrades. Testerna har en tydligt märkt engångsnyckel som bara används av testvärden i minnet.
Fel och oväntade situationer	Bra grund: 400 med <code>ProblemDetails</code> för ogiltigt <code>months</code> , laddnings- och felläge i frontend, skydd mot sena svar. Luckor i första granskningen: felmeddelandets format (F2) och en rubrik som kunde missförstås (F5). AI-tillfälle 2 hittade fler: ingen felgräns och ingen "Försök igen" (avsnitt 6).

Fynd som jag rättade

F1. Klockan går inte att styra (arkitektur och testbarhet)

AI föreslog `DateTime.UtcNow` direkt i `ForecastService`.

Problemet: resultatet beror på vilken dag koden körs, så månadsskiften, skottår och förfallna betalningar går inte att testa deterministiskt. AI:n skrev själv att testerna "bör räkna förväntade datum relativt dagens datum". Sådana tester ger olika utfall olika dagar, och de intressanta dagarna (31:a, 29 februari) kommer sällan.

Rättning: `TimeProvider` (inbyggd i .NET) injiceras. Produktionsbeteendet är oförändrat och testerna styr klockan.

Belägg: 23 nya tester, alla gröna mot AI:ns logik, så logiken var rätt. Jag tog sedan bort fyra saker i tur och ordning (filtret på aktiva prenumerationer, filtret på användare, "från idag"-gränsen och "räkna från ankardatumet"), och testerna larmade för alla fyra.

```
public class ForecastService(AppDbContext db, TimeProvider timeProvider) : IForecastService
{
    ...
    - var today = DateOnly.FromDateTime(DateTime.UtcNow);
    + var today = DateOnly.FromDateTime(timeProvider.GetUtcNow().UtcDateTime);
}
```

F2. Valideringen följer inte projektets mönster (indata)

AI föreslog gränserna 1–24 som konstanter i servicen, standardvärdet i kontrollern och ett handbyggt fel.

Problemet: projektets övriga indata valideras med `DataAnnotations` i en DTO. Här låg regeln på tre ställen, felnyckeln blev `months` i stället för `Months` och `Scalar` visade inga gränser.

Belägg: jag skrev testerna först, och de var **röda mot AI:ns version** (6 av 30).

Rättning: en `ForecastRequest` med `[Range]` och svenskt meddelande, bunden med `[FromQuery]`.

```
- ["months"] = ["Antal månader måste vara mellan {MinMonths} och {MaxMonths}."]
- public async Task<ActionResult<ForecastResponse>> Get(int months = DefaultMonths)
+ [Range(MinMonths, MaxMonths, ErrorMessage = "Antal månader måste vara mellan 1 och 24.")]
+ public int Months { get; init; } = DefaultMonths;
+ public async Task<ActionResult<ForecastResponse>> Get([FromQuery] ForecastRequest request)
```

Före: `{"errors": {"months": [...]}}`. Efter: `{"errors": {"Months": [...]}}`, som övriga endpoints, och OpenAPI anger nu minimum 1 och maximum 24.

F3. Arbetet växer med hur gammalt användarens datum är (säkerhet och prestanda)

AI föreslog att stega från `NextPaymentDate` fram till perioden.

Problemet: datumet kommer från användaren och saknade nedre gräns, så kostnaden växer med datumets ålder.

Belägg (uppmätt, median av 5 anrop mot det riktiga API:t, lokalt mot SQLite, så siffrorna är riktmärken): 300 veckoprenumerationer med nästa betalning 0001-01-01 tog 403 ms, mot 94 ms med dagens datum och 55 ms med 1990-01-01. Det är ungefär 1 ms extra CPU per prenumeration och anrop i värsta fallet. Låg allvarlighet, men billigt att stänga.

Rättning: `IntervalsBefore` räknar ut hur många hela intervall som kan hoppas över utan att passera en betalning på eller efter perioden, och loopen börjar där. Efter rättningen tog samma data 52 ms.

```
- var date = NextPaymentDate;
- for (var count = 1; date <= to; count++)
- {
-     if (date >= from) yield return date;
-     date = BillingInterval.AddIntervals(NextPaymentDate, count);
- }
+ for (var count = BillingInterval.IntervalsBefore(NextPaymentDate, from); ; count++)
+ {
+     var date = BillingInterval.AddIntervals(NextPaymentDate, count);
+     if (date > to) yield break;
+     if (date >= from) yield return date;
+ }
```

Att en felaktig överhoppning skulle *tyst* missa betalningar gjorde mig försiktig. Ett test jämför därför den nya koden med AI:ns ursprungliga loop som referens för 4 000 slumpade prenumerationer och fönster, samt äldsta möjliga datum för varje intervall. Jag provade fem medvetet trasiga varianter och fyra fångades. Den femte (kvartal avrundat uppåt) hoppar bara över månader före perioden och kan inte ändra resultatet, en *ekvivalent mutant*, så att den överlever är rätt. (Referensen byttes senare mot en helt oberoende, se avsnitt 6.)

F4 och F5

F4, onödig spårning: frågan är en ren läsfråga, så den fick `AsNoTracking()`. **F5, rubriken kunde missförstås:** kortet skrev "de kommande 6 månaderna", men API:t räknar hela kalendermånader från innevarande månad och den första bara från idag. Sent i en månad är perioden nästan en månad kortare. Kortet visar nu t.ex. "18 558 kr till och med mars 2027" (figur 7).

Vad AI:n gjorde rätt

Det är lika viktigt att säga vad som höll. **Datumlogiken var korrekt** (0 avvikelser både mot min egen oberoende beräkning och mot granskarens). **Åtkomstkontrollen var rätt.** Controllern var tunn, DTO:er och mappning följde mönstret och befintlig kod återanvändes. Frontendkortet passade resten av dashboarden. Det jag fick rätta handlade om testbarhet, konsekvens med projektet, robusthet mot ovanliga indata och tydlighet. Inget av det gav fel svar vid vanlig användning, och flera av dem blev tydliga först när jag testade, mätte eller körde appen.

Det jag först lät vara, och läget nu

Att granska är också att avgöra vad som inte ska ändras. Tre av punkterna ändrade jag mig om när AI-tillfälle 2 visade bevis jag inte hade.

Fynd	Läge efter båda granskningarna
Svaret är större än UI:t använder (betalningslistan)	Åtgärdat efter AI-tillfälle 2: listan är valfri och antalet prenumerationer begränsat.
"Idag" räknades i UTC	Åtgärdat efter AI-tillfälle 2: svensk tid i prognos, dashboard och betalningar.
Datum saknade gränser (500-fel vid år 9999)	Åtgärdat efter AI-tillfälle 2: datum mellan 2000 och 2100 i API och frontend.
Engelsk standardtext för icke-numeriska frågeparametrar (months=abc)	Kvarstår. Global modellbindning som alla endpoints delar, och frontend skickar bara 3, 6 eller 12.
ForecastChart liknar PaymentsChart	Kvarstår. Datan och tooltipen skiljer sig, och en abstraktion med två användare vore för tidig.
RegisterPayment flyttar fram datumet från föregående datum (31 → 28 och förblir 28)	Kvarstår, finns i befintlig kod. Rätt åtgärd är att lagra betalningsdagen, vilket kräver en migration. Kommentaren på GetPaymentDates var missvisande och är rättad.

6. AI-tillfälle 2: oberoende analys

Efter mina rättningar bad jag en ny agent med tomt sammanhang att granska koden som en kritisk senior utvecklare och säkerhetsgranskare. Den fick bara läsa koden, köra tester och egna skript, inte ändra något, och den fick *inte* läsa mina granskningsanteckningar.

Prompt (ordagrant)

Jag har byggt en prognosfunktion i min webapplikation SubTracker (ASP.NET Core 10 Web API + React/Vite). Repot ligger i /home/user/fullstack-app-azure-ai (läs README.md först). Funktionen visar vad användaren kommer att betala per månad framåt, utifrån aktiva prenumerationer.

Koden som ska granskas:

- Backend: backend/SubTracker.Api/Controllers/ForecastController.cs, Services/ForecastService.cs och IForecastService.cs, Dtos/Forecast/*, Mappings/ForecastMappings.cs, Entities/Subscription.cs (GetPaymentDates) och Entities/BillingIntervalExtensions.cs, samt registreringen i Program.cs.
- Frontend: frontend/src/api/ForecastApi.js, components/ForecastCard.jsx, components/charts/ForecastChart.jsx och hur de används i pages/DashboardPage.jsx.
- Tester: backend/SubTracker.Api.Tests.

Granska koden som en kritisk senior utvecklare och säkerhetsgranskare. Leta efter:

- buggar och felaktiga edge cases (datum, avrundning, tidszoner, tomma data)
- säkerhetsrisker (åtkomstkontroll, indatahantering, informationsläckage, resursmissbruk)
- arkitekturproblem och brott mot projektets egna mönster
- bristande felhantering
- prestandaproblem
- luckor i testerna, det vill säga beteenden som inget test skulle upptäcka om de gick sönder

Regler:

- Ändra ingen kod i repot och gör inga git-commits. Du får köra `dotnet build`/`dotnet test` och skriva egna tillfälliga skript utanför repot för att bevisa dina fynd.
- Läs inte mappen docs/ai/. Jag vill ha en oberoende bedömning.
- Rapportera bara fynd som du kan motivera. För varje fynd anger du allvarlighetsgrad (kritisk, hög, medel eller låg), fil och rad, vad som är fel, ett konkret scenario eller bevis och ett förslag på rättning.
- Skilj tydligt på fynd du har verifierat genom att köra något och sådant du bara misstänker.
- Skriv också vad du granskade och inte hittade något att anmärka på.

Svara på svenska.

Vad agenten gjorde

Agenten körde baslinjen (40 av 40 tester gröna), skrev 16 egna probetester, jämförde datumlogiken mot 285 892 fall i Python (0 avvikelser), körde 29 medvetet felaktiga kodvarianter (19 fångades, **10 överlevde**), drev appen i Chromium (nätverkstidslinjer, felscenarier, etikettmätning, XSS-försök), mätte svarsstorlek och minne i en fristående server och översatte frågan till T-SQL. Rapporten hade **12 fynd** (2 medel, 10 låga) och en lång lista över sådant den granskade utan anmärkning: åtkomstkontroll, indatahantering, SQL, aggregering, avrundning, XSS och informationsläckage.

Min bedömning av varje fynd

Jag verifierade varje fynd med en egen körning innan jag agerade. Hela rapporten och mina anteckningar finns i `docs/ai/03-ai-analys.md`.

#	AI:ns fynd	Min verifiering och bedömning	Åtgärd
F1	Medel Obegränsad resursförbrukning	Reproducerat: 1 000 försök att skapa prenumerationer gav $1\,000 \times 201$ och ett svar på 22 MiB. Azure-effekten har jag inte kunnat pröva.	Betalningslistan valfri och tak på 200 prenumerationer. Rate limiting, <code>CancellationToken</code> och komprimering valdes bort (globala beslut).
F2	Medel CI kör inga tester. 10 av 29 mutanter överlever	Körde om alla 29: samma 10 överlevare . Träffande: min referens återanvände <code>AddIntervals</code> och en kommentar påstod Python-verifiering utan att skriptet fanns i repot.	CI kör testerna före deploy, 16 nya tester, oberoende referens, skripten i repot.
F3, F11	Låg "Idag" i UTC, och <code>TimeProvider</code> bara i prognosen	Fem tester röda. Jag hade lämnat det i första granskningen ("dashboarden gör likadant"), men granskarens bevis att det syns oense för användaren fick mig att ändra mig.	<code>SwedishTimeProvider</code> , tre tjänster använder <code>GetToday()</code> .
F4	Låg <code>RegisterPayment</code> glider 31 → 28	Känt. Kommentaren var missvisande. Beteendet är ett befintligt designfel.	Kommentar rättad. Beteendet kvarstår (kräver migration).
F5	Låg Ingen felgräns: hela appen blir tom	Mitt första försök att återskapa misslyckades (musen utanför visningsytan). Sedan: sidans innehåll gick från 1 452 till 0 tecken.	<code>ErrorBoundary</code> runt kortet, null-säkert diagram.
F6	Låg Etiketter krockar på telefon	Bekräftat: 5 överlapp på 360 px. Granskarens åtgärd (dölja dem) var för trubbig .	Kortare belopp på smal skärm ("3,4 tkr").
F7	Låg Ingen "försök igen", inget avbrott, kaskadladdning	Bekräftat. Det ersatta 12-månadersanropet laddades ner klart.	"Försök igen" och <code>AbortController</code> . Kaskaden lämnades (medveten avvägning).
F8	Låg Inga datumgränser, 500 för år 9999	Hade jag själv verifierat. Hänger ihop med resursfrågan.	Datum mellan 2000 och 2100 i API och frontend.
F9	Låg Dashboard och prognos oense om förfallna	Förvirringen är verklig, beteendet avsiktligt.	Texten under diagrammet förtydligad.
F10	Låg Kulturberoende sortering	Bekräftat: olika ordning med ICU och i invariant läge. Granskarens förslag (<code>sv-SE</code>) är fortfarande olika utan ICU.	<code>OrdinalIgnoreCase</code> och id som tie-breaker.
F12	Låg Diagrammet saknar namn, laddning aviseras inte	Bekräftat i DOM.	Diagrammet har ett namn, <code>aria-busy</code> .

Där jag bedömde annorlunda, och vad rapporten inte kunde veta

- **F6:** att dölja etiketterna löser kollisionen men kostar information som oftast ryms. Jag mätte och valde kortare belopp. Mätt utan överlapp på 320–576 px, även för femsiffriga belopp.
- **F10:** granskarens jämförare löser inte det den pekar på, eftersom svensk sortering beror på ICU. Min egen körning: `OrderBy` ger `apple | Apple | Åsa | Banan | Zeta` med ICU men `Apple | Banan | Zeta | apple | Åsa` i invariant läge, medan en ordinal jämförelse ger samma ordning i båda.
- **F1:** jag tog de två åtgärder som begränsar värsta fallet, inte hela listan. Rate limiting och komprimering är beslut för hela API:t.
- **Azure-effekten i F1** är en misstanke: kvoterna är inte kontrollerade och ingenting är mätt mot Azure. Etikettmätningen gjordes med en bredare systemfont än på riktiga telefoner. Rapporten är inte helt konsekvent (89–100 % mot 97–100 % av svaret som aldrig används).

Två saker som gick snett hos mig

1. **Mitt första försök att återskapa F5 misslyckades.** Sidan blev inte tom, så fyndet såg först ut att vara fel. Orsaken var mitt eget test: musen hamnade utanför visningsytan och tooltipen öppnades aldrig. Med en hover som först scollar elementet i vy bekräftades kraschen, och sidans innehåll gick från 1 452 till 0 tecken. Hade jag avfärdat fyndet hade jag haft fel.
2. **Mina egna rättningar hade ett hål.** Testerna ersätter klockan, så den *riktiga* `SwedishTimeProvider` och dess registrering i `Program.cs` kördes aldrig. Granskaren kunde inte se det (den granskade koden före rättningarna), men jag hittade det genom att tillämpa dess metod, mutationsprovning, på mina egna rättningar, och lade till tester.

7. Test och slutverifiering

Automatiska tester

Testfil	Antal	Vad den skyddar
<code>ForecastEndpointTests</code>	29	401 utan token, endast egna prenumerationer, perioden över årsskiftet, 1–24 månader, felformat, förfallna och dagens betalningar, månadsskifte, skottdag, vecka/kvartal/år, inaktiva, kategorisummor och sortering, JSON-formen, svensk tid runt midnatt, valfri betalningslista
<code>SubscriptionPaymentDatesTests</code>	10	Datumlogiken mot en referens med egen kalenderaritmetik (4 000 slumpade fall, äldsta möjliga datum)
<code>BillingIntervalExtensionsTests</code>	10	<code>IntervalsBefore</code> hoppar aldrig över en betalning och lämnar högst ett steg kvar
<code>SubscriptionDateValidationTests</code>	9	Datumgränserna 2000–2100 och att "Markera betald" vid övre gränsen inte kraschar
<code>SwedishTimeProviderTests</code>	6	Den riktiga klockan och registreringen i <code>Program.cs</code>
<code>PaymentEndpointTests</code> , <code>ForecastServiceTests</code> , <code>DashboardEndpointTests</code>	12	"Markera betald" flyttar fram datumet som förut, servicens vakter och att läsfrågan inte spårar entiteter, baslinje (401, standardkategorier, summor, isolering)
<code>SwedishTimeTests</code> , <code>SubscriptionLimitTests</code>	4	Dashboard och betalning i svensk tid, taket på 200 prenumerationer
Backend totalt	80	Hela kedjan via HTTP mot SQLite i minnet (<code>WebApplicationFactory</code>)

Testfil	Antal	Vad den skyddar
ForecastCard.test, ErrorBoundary.test, validation.test, format.test	33	Laddning, fel, "Försök igen", avbrutna anrop, sena svar, aria-busy, felgränsen, datumreglerna, formatering (Vitest + Testing Library)

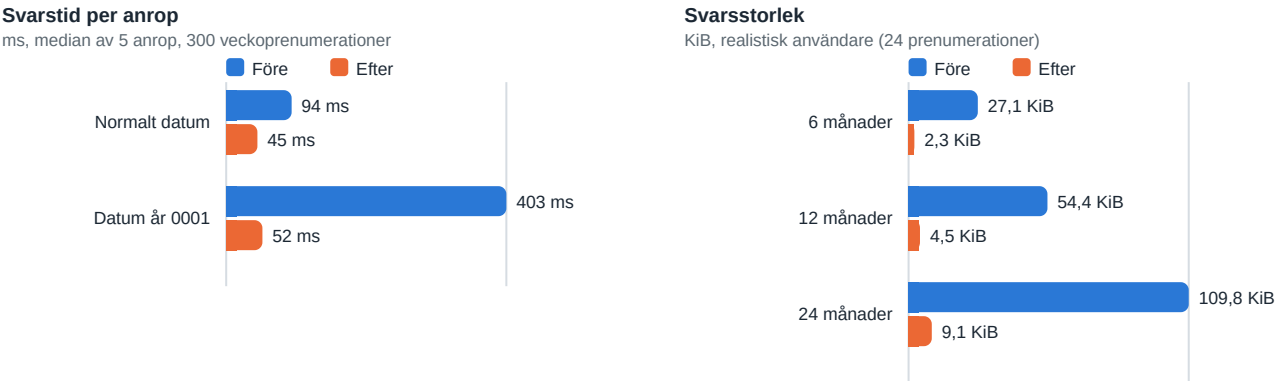
Både backend och frontend kör sina tester i CI före deploy. CI-sekvensen (`npm ci`, lint, test, build) har jag även kört i en ren kopia.

Mutationsprovning

Skriptet för in ett fel i taget i en kopia av koden och kör hela testsviten. **52 av 55 varianter fångas**. De tre som överlever är ekvivalenta, det vill säga de kan inte ändra något beteende som går att observera: två id-tie-breakers (ordningen blir ändå densamma, eftersom sorteringen är stabil och kategorinamn är unika per användare) och kvartal avrundat uppåt (hoppas bara över månader före perioden). Utskriften ligger i `docs/ai/verifiering/mutation_result.txt`.

Mängd	Före rättningarna	Efter
Granskarens 29 ursprungliga varianter (anpassade till den ändrade koden)	19 fångas, 10 överlever	29 fångas
Alla 55 varianter (inklusive varianter av varje rättning: taket, valfri lista, datumgränser, svensk tid)	–	52 fångas , 3 ekvivalenta

Mätningar

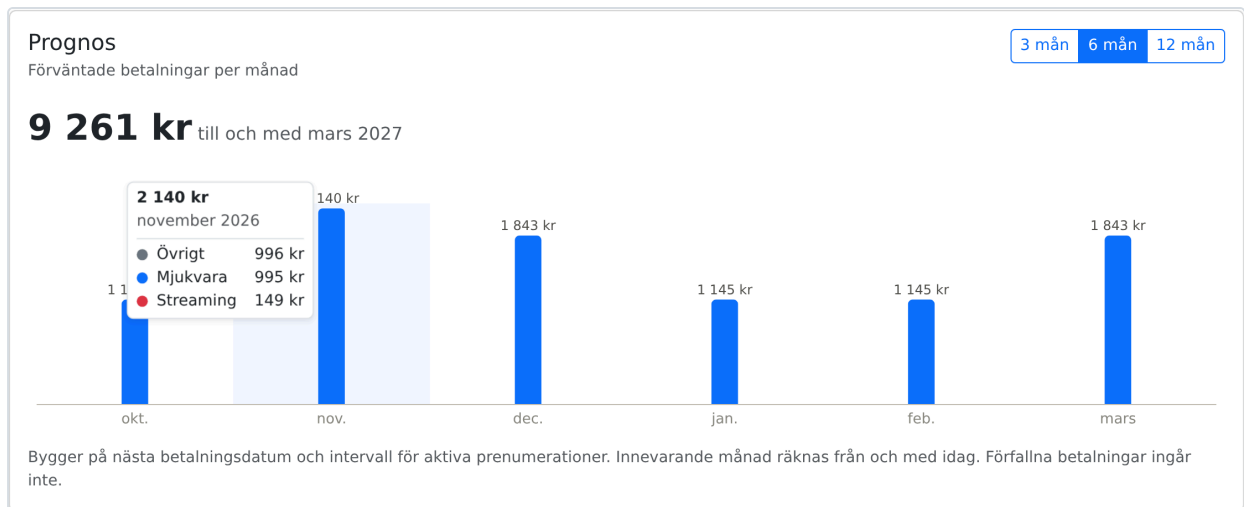


Figur 4. Före och efter. Vänster: svarstid för 300 veckoprenumerationer (lokalt mot SQLite, riktmärken). Förbättringen även för ett normalt datum kommer från `AsNoTracking`. Datum år 0001 mättes före datumgränserna, som nu gör att ett sådant datum inte kan skapas. Höger: svarsstorlek för en realistisk användare med 24 prenumerationer, före (svar med betalningslista, som AI-förslaget alltid skickade) och efter (standardsvaret, där listan är valfri). Mätt 4 oktober. Antalet betalningar beror på datumet, och en ny mätning 5 oktober gav 27,5, 55,5 och 111,7 KiB före och 2,3, 4,6 och 9,2 KiB efter.

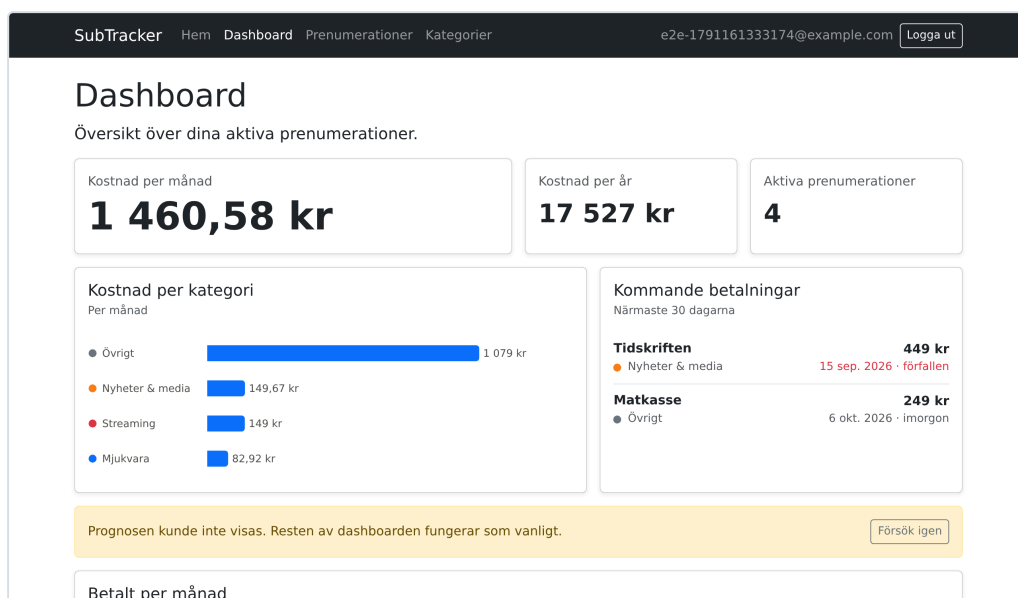
Scriptad användarresa i webbläsare

Ett skript registrerar en användare, lägger till fyra prenumerationer via formuläret (alla intervall, en förfallen), kontrollerar prognosens siffror mot en egen JavaScript-beräkning och provar felfall. **18 av 18 kontroller godkända**, och inga oväntade fel i webbläsarens konsol:

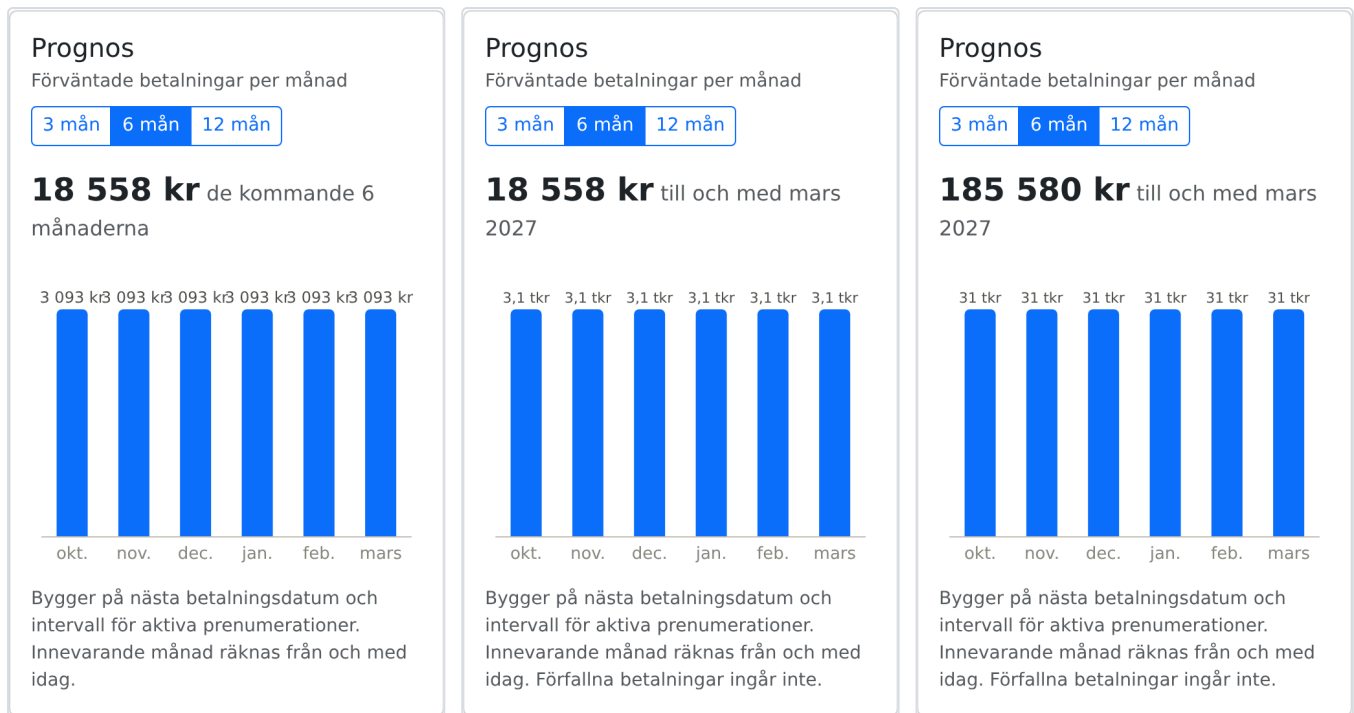
- Tomt läge för en ny användare. Formuläret nekar ett datum utanför 2000–2100 med ett tydligt meddelande.
- Standard 6 månader, samt 3 och 12: rätt summa, antal staplar och periodens slut. Tooltipen visar rätt månad, summa och kategorier.
- Webbappen ber inte om betalningslistan. "Markera betald" på Netflix ändrar prognosen som förväntat.
- Serverfel (500) visas i kortet med "Försök igen", som återhämtar kortet. Ett renderingsfel ersätter bara kortet (figur 6).
- På 360 px visas kortare belopp utan överlapp och utan horisontell scroll. Efter utloggning är den gamla token ogiltig (401) och dashboarden kräver inloggning.



Figur 5. Tooltipen visar månad, summa och fördelning per kategori. Månadens summa kontrolleras mot skriptets egen beräkning.



Figur 6. Ett renderingsfel i prognoskortet (framkallat med ett svar som bryter kontraktet, `months: null`) ersätter bara kortet med ett meddelande och "Försök igen". Navigeringen och resten av dashboarden finns kvar. Utan felgräns tömde ett renderingsfel hela appen (F5 i avsnitt 6).



Före: AI-förslaget.

Efter: slutversionen.

Efter, belopp $\times 10$.

Figur 7. Samma användare (sju månadsprenumerationer, 3 093 kr per månad, riktiga data) på 360 px. Vänster: AI-förslaget, där etiketterna rinner ihop ("3 093 kr3 093 kr...", 5 överlapp) och rubriken säger "de kommande 6 månaderna". Mitten: slutversionen (0 överlapp, periodens slut i rubriken, texten om att förfallna inte ingår). Höger: slutversionen med tio gånger så höga belopp (0 överlapp). Mätt med `label_overlap_check.mjs`.

Figur 8. Formuläret nekar ett datum utanför 2000–2100 direkt, med ett tydligt meddelande, i stället för att ge 500-fel vid "Markera betald".

Begränsningar

- Allt kördes mot SQLite, inte SQL Server. Frågan använder bara enkla filter som översätts likadant, och granskaren översatte den till T-SQL offline, men den är inte körd mot en riktig SQL Server.
- Webbläsarkörningen använder en systemfont i headless Chromium, som är bredare än typsnitten på riktiga telefoner, så etikettmätningarna är försiktiga.
- Azure-planens kvoter och minnesgränser är inte kontrollerade. Mätningarna av svarsstorlek och svarstid är lokala och indikativa.

8. Hur kvalitet, säkerhet och arkitektur kontrollerades

Område	Hur	Resultat
Kvalitet (rätt beteende)	Oberoende beräkning i Python och JavaScript, 80 + 33 automatiska tester, mutationsprovning (52 av 55), scriptad körning i webbläsare.	0 avvikelser, 18 av 18 kontroller
Säkerhet: åtkomst	Alla frågor filtreras på inloggad användare. Test där en användare försöker nå en annans data. Mutation som tar bort filtret (12 tester faller).	Rätt
Säkerhet: indata	<code>months</code> valideras i en DTO. Datum begränsade till 2000–2100. Antal prenumerationer begränsat. Betalningslistan valfri. XSS-försök mot namn, som React visar som text, gjordes av granskaren.	Rättat och testat
Säkerhet: resurser	Uppmätt arbete och svarsstorlek före och efter. Tak och valfri lista begränsar värsta fallet (22 MiB och växande → 4,4 MiB).	Begränsat
Secrets	Sökning i hela diffen. Inga konfigurationsfiler ändrades. Testnyckeln är märkt som engångsnyckel.	Inga
Arkitektur och placering	Jämförelse med projektets mönster: tunn controller, service bakom interface, DTO:er, mappning, logik på entiteten. Avvikelserna (validering i service, hårdkodad klocka) rättades.	Följer mönstret
Felhantering	400 med <code>ProblemDetails</code> , felläge och "Försök igen", avbrutna anrop, felgräns. Verifierat i webbläsare med medvetna serverfel och trasigt svar.	Rättat och testat

9. Bedömning av AI-verktygets bidrag

Vad verktyget hjälpte till med

- **Ett användbart förslag på en gång.** Agenten läste projektet, följde dess mönster och levererade backend och frontend som byggde och passade in. Granskningen krävde mer arbete än själva skrivandet.
- **Breddgranskning som är tidskrävande att göra för hand.** Den oberoende granskaren körde 29 mutanter, jämförde 285 892 datumfall, mätte nätverk, minne och etiketter i en riktig webbläsare och översatte frågan till T-SQL. Det hittade tidszonen, flera testluckor och resursförstärkningen.
- **Verktyg för verifiering.** Skript för oberoende beräkning, mutationsprovning och webbläsarkörning gick snabbt att få fram med AI och ligger nu i repot, så att de går att köra om.

Begränsningar och risker jag upplevde

- **AI:ns egen försäkran ersätter inte verifiering.** Förslaget "kompilerade och var testat", och var ändå svårt att testa på de punkter som spelade roll (klockan lästes direkt ur systemet). Att det var bra gjorde det lättare att tro på.
- **Blinda fläckar i det som inte efterfrågades.** Prompten sade ingenting om testbarhet, tidszoner eller resursförbrukning. AI:n nämnde UTC och beroendet av dagens datum i sin sammanfattning men löste dem inte, och resursfrågan togs inte upp alls.
- **Samma modellfamilj kan dela blindas fläckar.** Generatoren och granskaren är båda Claude. Att den andra hittade andra saker är lugnande, men det bevisar inte att allt hittades. Mina egna körningar var det som gav tilliten.
- **Rapporter kan vara nästan rätt.** Granskaren hade rätt i sak men inte alltid i lösningen (etiketter, sortering), och rapporten hade små inkonsekvenser. Utan egen verifiering hade jag antingen tagit in för mycket eller avfärdat något riktigt.

- **Skört och icke-beständigt.** En omstart av containern gjorde att en hel granskningskörning gick förlorad utan rapport. Allt som inte är committat och pushat kan försvinna.
- **Jag gör också fel.** Mitt eget första försök att återskapa F5 misslyckades, och mina egna rättningar hade ett testhåll. Granskning av AI-kod kräver samma noggrannhet mot sina egna steg.
- **Omdöme går inte att delegera.** Att avgöra vad som ska åtgärdas, vad som lämnas och vilken lösning som är rätt (kortare etiketter, ordinal sortering, tak i stället för rate limiting) var mina beslut, och de var inte självklara.

Vad jag skulle göra annorlunda

- Skriva **icke-funktionella krav** i prompten från början: testbarhet, tidszon, gränser på indata och svarsstorlek. Flera av fynden (klockan, tidszonen, gränserna på indata, svarsstorleken) hade då sannolikt kunnat undvikas redan i första förslaget.
- Be om **tester och en enkel hotmodell** redan i generationssteget, och sedan granska testerna lika noga som koden.
- Köra **mutationsprovning tidigt**. Den visade luckor som vanlig läsning och höga testantal inte visade, bland annat att 40 gröna tester lämnade 10 av 29 fel oupptäckta.
- Behandla allt AI:n säger om sin egen kod som **obevisat** tills jag har kört det.

Slutsats

AI var mest värdefull som **snabb författare och bred granskare**, inte som något man kan lita på utan kontroll. Resultatet blev bättre än förslaget var från början, och det beror på att varje påstående kontrollerades. Jag skulle använda AI på samma sätt igen, med mer krav i prompten och med verifieringen som en självklar del av arbetet.

Bilaga A. Commits som hör till uppgiften

Allt ligger i repot `HappyHamster135/Fullstack-App-Azure-Ai` på grenen `claude/blissful-edison-7izj8g`, ovanpå `a936831` (sista commit på `main` när projektet kopierades till repot). Listan hämtades från git när rapporten byggdes. Commit för själva rapporten och inlämningsfilerna anges i `inlamning.txt`.

Commit	Roll	Beskrivning
<code>f0b2903</code>	Förberedelse	Testprojekt med integrationstester mot SQLite
<code>ab8b68c</code>	AI-tillfälle 1: förslaget, orört	AI-förslag (ogranskat): prognos – backend
<code>00fa1c3</code>	AI-tillfälle 1: förslaget, orört	AI-förslag (ogranskat): prognos – frontend
<code>a818df7</code>	AI-tillfälle 1: logg	AI-logg 1: prompt och svar för kodgenereringen
<code>49e6664</code>	Egen granskning	Granskning 1: injicerbar klocka i prognosen + tester av beteendet
<code>a584ee1</code>	Egen granskning	Granskning 2: validera months som övrig indata (ForecastRequest)
<code>1622a70</code>	Egen granskning	Granskning 3: begränsa beräkningen av betalningsdatum + AsNoTracking
<code>69a54cb</code>	Egen granskning	Granskning 4: ange prognosperiodens slut i stället för "N månader"
<code>5041bb6</code>	Egen granskning: rapport	Granskningsrapport: fynd, belägg och rättningar för AI-förslaget
<code>f2d51d7</code>	AI-tillfälle 2: logg, ordagrant	AI-logg 2: prompt och oberoende granskningsrapport (ordagrant)
<code>6915380</code>	Åtgärd efter AI-tillfälle 2	Analys 2 (F2): CI kör testerna + testluckor som mutationsprovningen hittade
<code>d161168</code>	Åtgärd efter AI-tillfälle 2	Analys 2 (F3, F11): "idag" räknas i svensk tid i prognos, dashboard och betalningar
<code>5d69d13</code>	Åtgärd efter AI-tillfälle 2	Analys 2 (F1, F10): begränsa prognosens svarsstorlek, tak på prenumerationer, deterministisk sortering
<code>7b002d9</code>	Åtgärd efter AI-tillfälle 2	Analys 2 (F8, F4): datumgränser för prenumerationer, ärlig kommentar och kända begränsningar

Commit	Roll	Beskrivning
c53c86c	Åtgärd efter AI-tillfälle 2	Analys 2 (F5, F6, F7, F9, F12): robustare och tydligare prognoskort + frontendtester
37698f0	Åtgärd efter AI-tillfälle 2	Tester av den riktiga klockan: SwedishTimeProvider och registreringen i Program.cs
d66fed8	Verifiering	Verifieringsskript: oberoende beräkning, mutationsprovning, helkörning i webbläsare
81b0789	AI-tillfälle 2: min bedömning	AI-logg 2: min kritiska bedömning av granskningsrapporten
097072d	Dokumentation	Dokumentation: index över AI-loggen och hänvisning i README

Bilaga B. Köra verifieringen

Skripten ligger i `docs/ai/verifiering/` och beskrivs i den mappens README. Från repots rot:

```
dotnet test                                # 80 backendtester
cd frontend && npm test                     # 33 frontendtester

dotnet run --project docs/ai/verifiering/harness -- 5077 # API:t mot SQLite
cd frontend && VITE_API_URL=http://localhost:5077 npx vite --host 127.0.0.1
cd docs/ai/verifiering
python3 oracle_check.py                    # oberoende beräkning, 0 avvikelser
python3 mutation_check.py                  # 55 trasiga varianter, tar ca 10 minuter
node e2e_final.mjs                         # 18 kontroller i webbläsare
```